

Lab Assignment 3

Assignment Name: Experiments with Hybrid Video Encoding/Decoding Scheme

Due Date: 22 May, 2026. Submit in Moodle your report file in PDF and all the Matlab or notebooks/Python scripts with the code you have developed as a single compressed archive.

Purpose

This work intends to apply and consolidate knowledge about the elementary tools that are commonly used to implement the hybrid-based scheme for video compression, which form the basis of SoA video compression algorithms. The goal of this assignment is to implement a simplified hybrid video encoding scheme based on MPEG principles, including intra-frame (I-frame), predictive (P-frame), and bidirectional (B-frame) compression. Students will develop and integrate key encoding components separately and then combine them to compress and reconstruct a video sequence. Additionally, the impact of various encoding parameters on compression rate and quality will be analysed.

Why is it important that multimedia students gain the listed skills and knowledge? Because it provides them insights on the principles and practical skills of hybrid transform-based and perceptually-aware media compression, techniques which constitute the building blocks of the most up-to-date image and video compression algorithms and standards. Students will gain both theoretical knowledge and practical skills essential for careers in video coding, multimedia processing, and real-time streaming applications.

Skills

In this assignment, you'll be learning how to:

- 1) Using multimedia processing libraries (e.g., OpenCV, NumPy, FFmpeg or Matlab built-in functions)
- 2) Developing motion estimation and compensation algorithms
- 3) Implementing matrix-based DCT and quantisation
- 4) Designing and testing different GOP structures
- 5) Running experiments with different encoding parameters
- 6) evaluate the efficiency of the selected techniques.

Knowledge

In this assignment, you'll be learning about:

- 1) Understanding hybrid video encoding (I, P, B frames) and how different coding tools contribute to compression efficiency. Role of different GOP structures (e.g., IBBP, IPPP) in video compression
- 2) Trade-offs between compression efficiency and video quality, and the impact of GOP configuration
- 3) Concepts of motion estimation (e.g., full search, three-step search) and the role of motion compensation in predictive coding. Understanding macro-block structures and their impact on compression.
- 4) Balancing compression efficiency with perceptual video quality.

Task overview

The objective of this work is to allow students to acquire a better understanding of the principles of a hybrid video compression scheme, adopting a perceptually-aware approach, and how varying the value of different parameters in the different tools used affects efficiency.

To carry out these experiments, whenever necessary, use video sequences available on the UC's Moodle as input signals for the algorithms/processes implemented by the developed programs or scripts (some also available on the UC's Moodle page). It is intended that the student acquire a good understanding of the efficiency and adequacy of the different alternatives for compressing video sequences.

In some cases, you will be able to reuse scripts that you developed in assignment 2 or adopt similar approaches. Some example scripts can also be found in a corresponding folder in Moodle. Video sequences and individual images from video sequences are also available on Moodle. Examples of code and functions are also provided in a separate annex to this description.

Implementation work to be carried out

1. Experiments with hybrid video encoding

In these experiments, you will be compressing a video sequence applying compression tools typical of an MPEG-like hybrid video encoder (use only the luminance channel), balancing the use of each encoding mode (I, P, and B) along time and registering the compression degree. The most important compression tools of a hybrid video encoder are the bi-dimensional DCT, the motion estimation, the quantisation, and the statistical encoding tools. As such you will need to develop independent functions or scripts that implement each of these tools and the corresponding inverse operations. The statistical tools are optional.

1.1. Develop the following functions/scripts:

- i. Implementation of the basic functionality of importing a sequence of images, presenting it on the screen, and converting it to a “luminance/brightness + color” space (the decision to which color space to convert can be based on the value of a parameter passed to the script).
- ii. Computation of the motion vectors of an image on a macro-block basis, using one of the fast methods. The function needs to receive the anchor image or only the MB of the anchor image (the one to be encoded), the reference images (one or two depending on whether P or B mode is desired); a parameter indicating the encoding mode; a parameter indicating the pixel accuracy (default is one); a parameter indicating the window size (default is 15). The output should be the motion vector for the MB (pair of coordinates) and the difference MB.
- iii. Modified version of function ii, enabling the variation of the macro-block size, from 16x16 down to 4x4 pixel blocks. The function still receives the same parameters. You may fuse the two versions and build only one that receives an additional parameter indicating the minimum MB size to be used.
- iv. Function to plot the motion vectors computed for each MB.
- v. Application of the DCT2 transform to a block of 8x8 elements. The idea is to develop a function that can be invoked successively to apply the DCT2 transform to every 8x8 pixel block of the luminance component of an image. Note: You may reuse code developed in assignment 2.

- vi. Implementation of matrix-based quantisation of a block of 8x8 elements. The function defines a base quantisation matrix (use the values of the default quantisation matrices of MPEG - one to be used when the DCT is applied to full pixel values, i.e., in the I coding mode, and another one to be used when the DCT is applied to the difference or error values, i.e., for the P and B encoding modes). The function needs to receive the following input parameters: encoding mode; quantiser scale factor; block of 8x8 elements. A scaled quantisation matrix is used to divide the values of the 8x8 element block. Perform integer division. The idea is to invoke this function for each of the 8x8 coefficient blocks of a DCT transformed image.

1.2. Write a script that:

- Imports a video sequence using function 1.1.i
- For each image in the video sequence and according to a defined GOP structure, it divides the luminance channel of the image into units of (n x n) elements - 16x16 pixel blocks (the Macro Blocks, MB) or in 8x8 pixel blocks. The GOP structure can be passed as a parameter to the script (N, length, and M, number of B+P groups), or can be asked at runtime to the user, or can even be hardcoded. For each of the luminance units in the image, and according to the GOP structure, it appropriately invokes the developed functions in 1.1. The appropriate complete chain of functions should be applied on a unit basis to the whole image - once the first appropriate function has been applied to all of the units in the whole image, the next function is invoked in the same manner. The idea is to have at the input of this chain an original sequence of Y images and at the output a compressed version. Test with different types of sequences in terms of content (spatial and temporal complexity) and using different MB sizes, pixel accuracy and GOP structures. For the performed experiments, compute and compare the total amount of bits used. This script should store appropriately in disk files the compressed data (compressed image data and motion vectors).

2. Experiments with hybrid video decoding - Video Reconstruction and Performance Evaluation

In these experiments, you will be applying the MPEG-like decoding algorithm to the compressed luminance channel of a sequence of images and evaluating the effects on the quality of the recovered images according to the quantiser scale factor, MB size, and GOP structure.

2.1. Develop the following functions:

- i. function to perform inverse quantisation using the same base matrix and quality factor used in function 1.1.iii.
- ii. Application of the inverse DCT2 transform to a block of 8x8 elements. The idea is to develop a function that can be invoked successively to apply the inverse DCT transform to every quantised 8x8 coefficient block of an image.
- iii. Function to recover the full pixel data of a motion-compensated image within a compressed video sequence.
- iv. function to evaluate the quality of the recovered image (use PSNR and SSIM).

2.2. Write a script that:

- Imports a sequence of compressed images as obtained in 1.2.
- Implements the corresponding decoding process by appropriately invoking functions developed in 2.1. Displays the recovered video sequence and prints the quality for each image, indicating the

CIM 2025-2026
M.EEC0045

parameters that were used for the encoding process (motion vector parameters, quantiser scale factor, GOP structure). Run the experiment for every encoded sequence obtained in 1.2.

Criteria for Success

It is important that you familiarise yourself with the combined use of compression tools to build a complete chain for video encoding. Also, you should understand that it is possible to control the efficiency of the compression process (quality vs bit rate) by manipulating the values of certain parameters of tools or

Criteria	Description	Weight
Implementation	Correctness and completeness of encoding steps	50%
Analysis	Depth of parameter impact analysis	30%
Code Quality	Readability, structure, and documentation	10%
Report Quality	Clarity, organization, and results presentation	10%

the way tools are combined.

- **Assessment criteria**

Report should be delivered up to 22 May in Moodle.